```
{
    printk(KERN_INFO "Driver: close()\n");
    return 0;
}
static ssize_t my_read(struct file *f, char __user *buf, size_t
len, loff_t *off)
{
    printk(KERN_INFO "Driver: read()\n");
    return 0;
}
static ssize_t my_write(struct file *f, const char __user *buf,
size_t len, loff_t *off)
{
    printk(KERN_INFO "Driver: write()\n");
    return len;
}

static struct file_operations pugs_fops =
{
    .owner = THIS_MODULE,
    .open = my_open,
    .release = my_close,
    .read = my_read,
    .write = my_write
};

static int __init ofcd_init(void) /* Constructor */
{
    printk(KERN_INFO "Namaskar: ofcd registered");
    if (alloc_chrdev_region(&first, 0, 1, "Shweta") < 0)
    {
        return -1;
    }
    if ((cl = class_create(THIS_MODULE, "chardrv")) == NULL)
    {
        unregister_chrdev_region(first, 1);
        return -1;
    }
    if (device_create(cl, NULL, first, NULL, "mynull") == NULL)
    {
        class_destroy(cl);
        unregister_chrdev_region(first, 1);
        return -1;
    }

    cdev_init(&c_dev, &pugs_fops);
    if (cdev_add(&c_dev, first, 1) == -1)
    {
        device_destroy(cl, first);
        class_destroy(cl);
        unregister_chrdev_region(first, 1);
        return -1;
    }
    return 0;
}

static void __exit ofcd_exit(void) /* Destructor */
{
    cdev_del(&c_dev);
    device_destroy(cl, first);
    class_destroy(cl);
    unregister_chrdev_region(first, 1);
    printk(KERN_INFO "Alvida: ofcd unregistered");
}

module_init(ofcd_init);
module_exit(ofcd_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Anil Kumar Pugalia <email@sarika-pugs.com>");
MODULE_DESCRIPTION("Our First Character Driver");
```

Shweta repeated the usual build process, with some new test steps, as follows:

1. Build the driver (.ko file) by running *make*.
2. Load the driver using *insmod*.
3. List the loaded modules using *lsmod*.
4. List the major number allocated, using *cat /proc/devices*.
5. 'null driver'-specific experiments (refer to Figure 2 for details).
6. Unload the driver using *rmmod*.

Summing up

Shweta was certainly happy; all on her own, she'd got a character driver written, which works the same as the standard */dev/null* device file. To understand what this means, check the <major, minor> tuple for */dev/null*, and similarly, also try out the *echo* and *cat* commands with it.

However, one thing began to bother Shweta. She had got her own calls (*my_open*, *my_close*, *my_read*, *my_write*) in her driver, but wondered why they worked so unusually, unlike any regular filesystem calls. What was unusual? Whatever was written, she got nothing when reading—unusual, at least from the regular file operations' perspective. How would she crack this problem? Watch out for the next article. END

By: Anil Kumar Pugalia

The author is a freelance trainer in Linux Internals, Linux Device Drivers, Embedded Linux and related topics. Prior to this, he was at Intel and Nvidia. He has been working on Linux since 1994. A gold medallist from the Indian Institute of Science, Linux and knowledge sharing are two of his many passions. More information about him can be found at *http://profession.sarika-pugs.com*, including his various Linux-related training sessions and workshops. He can be reached at *email@sarika-pugs.com*.